

EWAT

Early Warning and Anomaly Typing

Formalisation Mathématique Complète

Wassim Badraoui

Devoteam Research

8 avril 2026

Contexte. Stage de recherche *Root Cause Analysis*. Architectures microservices Kubernetes. Infrastructure : cluster Kubernetes 6 nœuds (HA). Télémétrie : OpenTelemetry (OTLP Collector unifié). Chaos engineering : Chaos Mesh, Locust.

Résumé. Ce document formalise le système EWAT, une contribution de recherche visant la détection précoce et le typage automatique des anomalies dans les architectures microservices Kubernetes. Le problème central est la séparation explicite des drifts bénins (déploiements, autoscaling) des anomalies réelles, préalablement à l'apprentissage d'une ontologie empirique des types de pannes. La contribution est évaluée par trois hypothèses falsifiables indépendamment.

Table des matières

Nomenclature et Acronymes	2
1 Modélisation du Système	3
1.1 Le graphe de services	3
1.1.1 Alternatives considérées	3
1.1.2 Fonction d'agrégation intra-service	3
1.1.3 Sémantique des arêtes	3
1.2 Le signal de télémétrie	4
1.2.1 Métriques (OpenTelemetry Metrics)	4
1.2.2 Features de traces (OpenTelemetry Traces)	4
1.2.3 Features de logs (OpenTelemetry Logs)	4
1.2.4 Bilan dimensionnel et tractabilité	4
1.3 Modélisation du régime opérationnel	5
2 Le Problème	5
2.1 Deux pathologies documentées	5
2.2 Définition de la tâche	5
3 Hypothèses de Recherche	6
3.1 H1 — Structurabilité	6
3.2 H2 — Séparabilité du drift	6
3.3 H3 — Prédicibilité	6
4 Solution : Pipeline EWAT	6
4.1 Étape 0 : Détection de drift multivariée en temps linéaire	6
4.1.1 Motivation	6
4.1.2 Solution : MMD-RFF	7
4.1.3 Filtrage avec look-through	7
4.2 Étape 1 : Encodeur spatio-temporel sur graphe stabilisé	7
4.3 Étape 2 : Typage par apprentissage contrastif faiblement supervisé	7
4.3.1 Signal de supervision	7
4.3.2 Apprentissage de la distance métrique	8
4.3.3 Interprétabilité	8
4.4 Étape 2b : Construction de l'ontologie	8
4.5 Étape 3 : Apprentissage des précurseurs par type	8
4.6 Sortie : Alerte EWAT	9
4.7 Budget de latence	9
5 Protocole d'Évaluation	9
5.1 Falsification des hypothèses	9
5.2 Justification du seuil de silhouette	9
5.3 Ablation des features	10
6 Synthèse des Contributions	10
6.1 Positionnement dans la littérature	11
6.2 Références	11

Nomenclature et Acronymes

EWAT	E arly W arning and A nomaly T yping. Le système décrit dans ce document, visant à anticiper et classifier les pannes avant qu'elles ne se produisent.
RCA	R oot C ause A nalysis. Analyse de la cause racine, une tâche <i>post-mortem</i> (comprendre pourquoi la panne a eu lieu).
SRE	S ite R eliability E ngineer. Ingénieur garant de la fiabilité des systèmes en production, utilisateur final du système EWAT.
Drift (bénin)	Modification légitime et attendue du comportement du système (ex: mise à jour, autoscaling) qui modifie la distribution des métriques sans constituer une anomalie.
STGCN	S patio- T emporal G raph C onvolutional N etwork. Réseau de neurones sur graphe capable d'apprendre des motifs à la fois dans l'espace (topologie du réseau) et dans le temps (évolution des métriques).
MMD-RFF	M aximum M ean D iscrepancy approximé par R andom F ourier F eatures. Méthode mathématique permettant de mesurer la distance entre deux distributions multivariées en temps linéaire.
Chaos Mesh	Outil d' <i>Engineering du Chaos</i> pour Kubernetes, utilisé ici pour injecter volontairement des pannes de manière contrôlée afin de générer les labels d'apprentissage.
AUROC	A rea U nder the R eceiver O perating C haracteristic curve. Métrique statistique évaluant la capacité du modèle prédictif à distinguer correctement les états normaux des anomalies, quel que soit le seuil d'alerte choisi.
Entropie de Transfert	<i>Transfer Entropy</i> . Outil mathématique issu de la théorie de l'information permettant de mesurer une relation de causalité asymétrique entre deux variables dans le temps, sans exiger que le système soit stationnaire (contrairement à la causalité de Granger).
Gray failure	Panne "grise" ou silencieuse. Dégradation partielle d'un système (par exemple, un seul réplica ralenti ou retournant des erreurs par intermittence) qui échappe aux seuils d'alerte globaux classiques basés sur des moyennes.
Indice de Silhouette	Mesure évaluant la qualité d'un partitionnement de données (clustering). Un score élevé (proche de 1) indique que les épisodes sont bien regroupés avec leurs semblables et clairement séparés des autres types d'anomalies.
Manifold shift	Changement structurel profond de la distribution des données. Contrairement à une simple variation univariée, un <i>manifold shift</i> modifie la topologie même du système et les corrélations entre les différentes métriques.
P99	99 ^{ème} percentile. Mesure de latence indiquant le temps de réponse maximum expérimenté par les 1 % des requêtes les plus lentes. C'est un indicateur crucial pour détecter les dégradations précoces de performance.
SHAP	S Hapley A dditive e x P lanations. Méthode issue de la théorie des jeux permettant d'expliquer les prédictions d'un modèle d'IA "boîte noire" en calculant la contribution exacte de chaque métrique à l'alerte générée.

1. Modélisation du Système

1.1 Le graphe de services

L'infrastructure cloud-native est modélisée par un graphe orienté dynamique pondéré :

$$G(t) = (V, E(t), w_E(t)) \quad (1)$$

où V représente les Services et Deployments Kubernetes. Les Pods, éphémères par nature, sont exclus de V : leurs créations (changement d'IP, nouvel identifiant) rendraient $|V|$ variable au sein d'une même fenêtre temporelle, ce que les GCN gèrent mal. L'agrégation au niveau Service / Deployment stabilise $|V| = N$ constant.

1.1.1 Alternatives considérées

Un graphe hiérarchique $G = (G_{\text{infra}}, G_{\text{service}}, G_{\text{namespace}})$ à plusieurs niveaux (Node $\rightarrow$ Pod $\rightarrow$ Service $\rightarrow$ Namespace) capturerait davantage de structure, mais réintroduirait l'instabilité de $|V|$ au niveau Pod et multiplierait la complexité du STGCN de façon disproportionnée pour un cluster de 6 nœuds. Cette extension reste une piste pertinente pour des clusters de grande envergure ($N > 50$).

Un graphe orienté ressources (CPU, volumes, ConfigMaps comme nœuds) est également écarté : ces entités ne correspondent pas aux unités de déploiement et rendraient l'interprétabilité SRE difficile, les alertes ne pointant plus vers des services identifiables dans les runbooks existants.

1.1.2 Fonction d'agrégation intra-service

Chaque service $v \in V$ peut contenir $R_v(t) \geq 1$ replicas à l'instant t . Le signal brut au niveau pod $s_r(t) \in \mathbb{R}^d$ est agrégé en un signal service par une fonction différenciée selon la nature de la métrique :

$$S_v(t) = \text{Agg}(\{s_r(t)\}_{r=1}^{R_v(t)}) \quad (2)$$

L'opérateur Agg est défini composante par composante. Les métriques de saturation (CPU, RAM, saturation réseau, disk I/O) utilisent le max, afin de ne pas diluer un replica en état dégradé dans la moyenne. Cela garantit la sensibilité aux *gray failures*, où un unique replica défaillant peut échapper à un moyennage naïf. Les métriques de taux (taux d'erreur, taux de warnings) utilisent une somme pondérée par le volume : $\sum_r n_r \cdot \text{taux}_r / \sum_r n_r$. Les métriques de latence (P99, durée médiane) utilisent le percentile 99 sur l'union des distributions des replicas, et non le percentile des percentiles, qui introduirait un biais systématique. Les métriques structurelles (profondeur de trace, fan-out, entropie lexicale) utilisent la médiane.

1.1.3 Sémantique des arêtes

Les arêtes $E(t)$ évoluent dynamiquement selon les patterns d'appel observés dans les traces Jaeger. Chaque arête porte un vecteur de features :

$$w_E(t) : E(t) \rightarrow \mathbb{R}^3, \quad e_{ij}(t) \mapsto (\text{volume}_{ij}(t), \text{latence_med}_{ij}(t), \text{taux_erreur}_{ij}(t)) \quad (3)$$

Ces poids permettent au GCN de moduler la propagation d'information : une arête à haut volume et haute latence propagera davantage le signal d'anomalie qu'une arête à faible volume. Le seuil de présence d'une arête est $\text{volume}_{ij}(t) > 0$ sur la fenêtre glissante.

1.2 Le signal de télémétrie

À chaque instant t , le système produit un signal multimodal agrégé au niveau Service. L'utilisation d'OpenTelemetry standardise la collecte et garantit l'alignement natif des trois modalités via la propagation de contexte (*TraceID* partagé) :

$$S(t) \in \mathbb{R}^{N \times d}, \quad S(t) = [M(t) \mid T(t) \mid L(t)] \quad (4)$$

1.2.1 Métriques (OpenTelemetry Metrics)

$M(t) \in \mathbb{R}^{N \times d_M}$ avec $d_M = 7$ dimensions extraites des conventions sémantiques OTel : utilisation CPU, utilisation RAM, latence P99, taux d'erreur HTTP (4xx + 5xx), saturation réseau, disk I/O (IOPS et throughput), longueur de file d'attente (pending requests, queue depth).

Le disk I/O capture une classe de pannes (remplissage de PersistentVolume, log storms) qui ne se manifeste ni dans le CPU ni dans la RAM. La longueur de file d'attente est le *leading indicator* le plus précoce de saturation dans la méthodologie USE (Utilization, Saturation, Errors) de Gregg (2013), et constitue un signal critique pour un système d'early warning : la file sature avant que la latence ne se dégrade visiblement.

1.2.2 Features de traces (OpenTelemetry Traces)

$T(t) \in \mathbb{R}^{N \times d_T}$ avec $d_T = 6$ dimensions dérivées des spans OTLP : durée médiane des spans, taux de spans anormaux, profondeur de trace, fan-out, taux de retry, variance de latence (coefficient de variation).

Le taux de retry (ratio spans retentés / total) est un précurseur classique de cascade failure : les retries amplifient la charge en amont et transforment une dégradation locale en surcharge systémique. La variance de latence complète la durée médiane : un service dont la médiane est stable mais dont la variance explose est en dégradation progressive, invisible à la seule médiane.

1.2.3 Features de logs (OpenTelemetry Logs)

$L(t) \in \mathbb{R}^{N \times d_L}$ avec $d_L = 4$ dimensions : taux d'erreurs (lignes **ERROR** / total), taux de warnings, anomalie sémantique, entropie lexicale.

La dimension d'anomalie sémantique requiert une définition précise. Pour chaque service v , un corpus de référence $\mathcal{L}_v^{\text{ref}}$ est constitué à partir des logs en régime θ_{normal} . Chaque ligne de log ℓ est projetée via un modèle d'embedding de phrases pré-entraîné (Sentence-BERT) :

$$e(\ell) = \text{SentenceBERT}(\ell) \in \mathbb{R}^{384} \quad (5)$$

Le centroïde normal est $\mu_v = \frac{1}{|\mathcal{L}_v^{\text{ref}}|} \sum_{\ell \in \mathcal{L}_v^{\text{ref}}} e(\ell)$, et le score d'anomalie sémantique à l'instant t est :

$$a_v^{\text{sem}}(t) = \frac{1}{|\mathcal{L}_v(t)|} \sum_{\ell \in \mathcal{L}_v(t)} \|e(\ell) - \mu_v\|_2 \quad (6)$$

Ce score détecte l'apparition de messages inédits (stack traces nouvelles, erreurs jamais observées) sans dépendre de patterns regex manuels.

1.2.4 Bilan dimensionnel et tractabilité

$$d = d_M + d_T + d_L = 7 + 6 + 4 = 17 \quad (7)$$

Ce choix de $d = 17$ impacte l'étape 0 : le MMD-RFF opère dans un espace de dimension $d \cdot N$. Pour $N = 20$ services, l'espace est de dimension 340. L'approximation RFF requiert $D \gg \sqrt{d \cdot N}$ pour être fidèle au kernel gaussien ; la règle empirique $D \geq 10\sqrt{d \cdot N} \approx 185$ donne un surcoût qui reste linéaire en D , mais la qualité de l'approximation doit être validée expérimentalement. Le protocole d'évaluation (Section 5) inclut une expérience d'ablation pour quantifier la contribution de chaque feature et identifier les dimensions éventuellement redondantes.

1.3 Modélisation du régime opérationnel

Hypothèse 1.1 (Non-additivité du signal). Le signal $S(t)$ n'est pas une somme additive de composantes indépendantes. Une décomposition linéaire $S = S_{\text{normal}} + S_{\text{drift}} + S_{\text{anomaly}}$ supposerait une additivité incorrecte en présence de corrélations inter-métriques non linéaires.

Le système évolue sur une variété paramétrée par son état opérationnel $\theta(t)$:

$$S(t) \sim \mathcal{D}_{\theta(t)}(G(t)) \quad (8)$$

Un changement de régime est un *manifold shift* : il modifie la distribution tout entière, y compris la matrice de covariance inter-métriques et la topologie $E(t)$.

L'état $\theta(t)$ appartient à l'un de quatre régimes :

$$\theta(t) \in \{ \theta_{\text{normal}}, \theta_{\text{drift}}, \theta_{\text{anomaly}}, \theta_{\text{drift} \cap \text{anomaly}} \} \quad (9)$$

Le régime θ_{drift} encode les transitions bénignes (déploiements continus, autoscaling, maintenances planifiées) qui modifient légitimement $G(t)$ et la distribution de $S(t)$ sans constituer des anomalies. Le quatrième régime $\theta_{\text{drift} \cap \text{anomaly}}$ modélise les situations où un drift et une anomalie se superposent temporellement : un déploiement défectueux constitue simultanément une modification du système et un comportement pathologique. Ce cas, fréquent en production, est traité par un mécanisme de *look-through* à l'étape 0 du pipeline (Section 4.1).

2. Le Problème

2.1 Deux pathologies documentées

Aucun système actuel ne sépare explicitement θ_{drift} de θ_{anomaly} avant d'apprendre la structure des types d'anomalies. Ce mélange produit deux pathologies empiriquement documentées.

Les transitions θ_{drift} déclenchent des **faux positifs massifs** sur des systèmes entraînés uniquement sur des données saines : c'est la *benign drift* formalisée par Myrtollari et al. (2025) et Hinder et al. (2024).

Le **gap benchmark / production** est tout aussi problématique : les benchmarks simplifiés présentent des anomalies dont le signal est dominant sur le nœud source, alors qu'en production ce signal est noyé dans le bruit du drift. Fu et al. (2025) documentent un Top@1 de 0.21 en conditions réelles contre 0.63 sur benchmarks publics.

2.2 Définition de la tâche

Ce travail ne fait pas du RCA. Le Root Cause Analysis est une tâche post-mortem : *Où* et *Pourquoi*, après la panne. Ce travail produit un système d'Early Warning and Anomaly Typing : *Quoi* et

Dans combien de temps, avant la panne. Ces deux tâches sont distinctes dans la littérature et ne doivent pas être confondues lors de l'évaluation.

3. Hypothèses de Recherche

Hypothèse 3.1 (Hypothèse principale H). L'espace des anomalies dans un cluster Kubernetes forme une variété de basse dimension, structurée en types distincts, séparable des drifts bénins, et caractérisée par des précurseurs stables à horizon $T - k$, appris par apprentissage contrastif faiblement supervisé.

H se décompose en trois sous-hypothèses testables indépendamment.

3.1 H1 — Structurabilité

Il existe une partition $\mathcal{C} = \{C_1, \dots, C_K\}$ de l'espace des épisodes sous régime θ_{anomaly} telle que les épisodes d'un même type partagent une signature spatio-temporelle discriminante, apprise par apprentissage contrastif faiblement supervisé via les labels synthétiques des injections Chaos Mesh.

Critère de falsification : indice de silhouette < 0.3 en held-out sur 5 graines indépendantes avec splits de données distincts (justification en Section 5.2).

3.2 H2 — Séparabilité du drift

Les manifold shifts θ_{drift} sont géométriquement séparables des shifts θ_{anomaly} dans l'espace des embeddings, de façon multivariée et respectueuse de la structure de covariance inter-métriques du graphe.

Critère de falsification : absence de réduction statistiquement significative du taux de faux positifs après filtrage, à rappel constant sur anomalies réelles ($p > 0.05$, test de Student).

3.3 H3 — Prédicibilité

Pour chaque type $C_i \in \mathcal{C}$, il existe une signature précurseur observable à horizon $k > 0$ avant l'incident, dont l'AUCROC surpasse statistiquement une baseline de détection générique non typée.

Critère de falsification : AUCROC par type inférieur à la baseline sur l'ensemble des horizons k testés.

Remarque 3.1. H est falsifiable dans son ensemble : si $K = 1$ (aucune structure), si les clusters chevauchent θ_{drift} , ou si aucun précurseur n'est prédictif au-delà du hasard, l'hypothèse est rejetée. Ce rejet constitue lui-même une contribution publiable.

4. Solution : Pipeline EWAT

Le pipeline se déploie en quatre étapes séquentielles, chacune testable indépendamment.

4.1 Étape 0 : Détection de drift multivariée en temps linéaire

4.1.1 Motivation

ADWIN est un algorithme univarié. Appliqué indépendamment sur chaque métrique d'un système corrélé, il détruirait la covariance spatiale inter-métriques. Lors d'un autoscaling légitime, CPU

et RAM montent conjointement ; un filtre univarié les traiterait en décalé et produirait une décorrélation artificielle interprétée comme anomalie. De plus, le MMD standard a une complexité quadratique $O(n^2)$, incompatible avec un flux continu à haute fréquence.

4.1.2 Solution : MMD-RFF

On adopte le Maximum Mean Discrepancy multivarié approximé par Random Fourier Features, garantissant une complexité linéaire $O(nD)$ avec $D \ll n$:

$$\widehat{\text{MMD}}^2(W_{\text{ref}}, W_{\text{cur}}) = \left\| \frac{1}{|W_{\text{ref}}|} \sum_{x \in W_{\text{ref}}} \varphi(x) - \frac{1}{|W_{\text{cur}}|} \sum_{z \in W_{\text{cur}}} \varphi(z) \right\|^2 \quad (10)$$

où $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}^D$ est la projection RFF. Le test opère sur les embeddings du graphe entier, préservant la covariance spatiale.

4.1.3 Filtrage avec look-through

Le signal filtré intègre un mécanisme de look-through pour traiter la superposition $\theta_{\text{drift} \cap \text{anomaly}}$:

$$\tilde{S}(t) = \begin{cases} S(t) & \text{si } \widehat{\text{MMD}}^2 < \epsilon_{\text{drift}} \\ S(t) \text{ avec flag DRIFT} & \text{si } \widehat{\text{MMD}}^2 \geq \epsilon_{\text{drift}} \text{ et test post-drift positif} \\ \text{RECALIBRATE} & \text{si } \widehat{\text{MMD}}^2 \geq \epsilon_{\text{drift}} \text{ et test post-drift négatif} \end{cases} \quad (11)$$

Lorsque le MMD dépasse le seuil, un second test est appliqué sur la fenêtre $[t, t + \delta_{\text{post}}]$ pour déterminer si le nouveau régime est stable ou présente des marqueurs d'anomalie. Le cas RECALIBRATE met à jour $W_{\text{ref}} \leftarrow W_{\text{cur}}$: la baseline absorbe le nouveau régime. Le cas *flag DRIFT* transmet le signal aux étapes suivantes avec un marqueur indiquant qu'un drift concurrent est en cours, permettant à l'encodeur de conditionner son embedding.

Le seuil ϵ_{drift} est calibré empiriquement via injection de drifts bénins contrôlés (déploiements et autoscaling Chaos Mesh).

4.2 Étape 1 : Encodeur spatio-temporel sur graphe stabilisé

L'encodeur produit un embedding par épisode anormal sur une fenêtre $[t - W, t + \delta]$ centrée sur l'incident :

$$z_e = \text{Enc}_\theta(\tilde{S}_{[t-W, t+\delta]}, G(t)) \in \mathbb{R}^{d_e} \quad (12)$$

Le STGCN des travaux précédents est réutilisé directement. L'agrégation au niveau Service / Deployment garantit N constant quelle que soit la dynamique des Pods sous-jacents. Les arêtes pondérées $w_E(t)$ (Section 1.1) sont intégrées dans la convolution de graphe via une matrice d'adjacence pondérée.

4.3 Étape 2 : Typage par apprentissage contrastif faiblement supervisé

4.3.1 Signal de supervision

Deux épisodes issus de la même injection Chaos Mesh constituent une paire positive ; deux épisodes d'injections différentes constituent une paire négative. Ce signal est faiblement supervisé par les labels synthétiques contrôlés.

4.3.2 Apprentissage de la distance métrique

Un réseau siamois apprend une distance d_φ sur l'espace des embeddings :

$$d_\varphi(z_i, z_j) \rightarrow 0 \quad \text{si même type injecté} \quad (13)$$

$$d_\varphi(z_i, z_j) \rightarrow 1 \quad \text{sinon} \quad (14)$$

Sur cette métrique, un clustering hiérarchique agglomératif produit la taxonomie $\mathcal{C} = \{C_1, \dots, C_K\}$.

4.3.3 Interprétabilité

Pour chaque type C_i , les features discriminantes sont extraites par SHAP sur les embeddings, produisant une fiche interprétable directement lisible par un SRE sans supposer une correspondance préalable avec les types connus de la littérature.

4.4 Étape 2b : Construction de l'ontologie

La taxonomie seule n'est pas une ontologie. Trois types de relations sont construits explicitement entre types, formant un graphe orienté étiqueté :

$$\mathcal{O} = (\mathcal{C}, R), \quad R \subseteq \mathcal{C} \times \mathcal{C} \times \{\text{temporelle, causale, co-occurrence}\} \times \mathbb{R} \quad (15)$$

Relations temporelles. $C_i \xrightarrow{\Delta t, \sigma} C_j$ si C_i précède C_j avec un délai médian Δt et un écart-type σ statistiquement stables sur l'ensemble des incidents.

Relations causales. Le test de Granger est inadapté : il repose sur l'hypothèse de stationnarité, or le système évolue sous manifold shifts. On adopte l'Entropie de Transfert (Transfer Entropy), qui ne suppose pas la stationnarité :

$$TE_{C_i \rightarrow C_j} = \sum p(c_j^{t+1}, c_j^t, c_i^t) \log \frac{p(c_j^{t+1} | c_j^t, c_i^t)}{p(c_j^{t+1} | c_j^t)} \quad (16)$$

Une relation causale est ajoutée si $TE_{C_i \rightarrow C_j}$ dépasse un seuil de significativité estimé par permutation. L'estimation des distributions jointes conditionnelles utilise l'estimateur k -NN de Kraskov, Stögbauer et Grassberger (2004), non paramétrique et adaptatif en résolution, documenté comme plus robuste en petits échantillons qu'un estimateur histogramme. Le nombre minimum d'épisodes par paire (C_i, C_j) pour un test de significativité fiable est fixé à $n_{\min} = 30$; en deçà, la relation n'est pas testée.

Relations de co-occurrence. $C_i \leftrightarrow C_j$ si les deux types apparaissent simultanément plus souvent que l'indépendance statistique ne l'expliquerait (test du χ^2 sur la table de contingence).

4.5 Étape 3 : Apprentissage des précurseurs par type

Pour chaque type $C_i \in \mathcal{O}$, un modèle de précurseur f_i est entraîné sur la fenêtre précédant l'incident :

$$\hat{p}_i(t) = f_i(\tilde{S}_{[t-k, t]}, G(t)) \in [0, 1], \quad k \in \{2, 5, 10, 20, 30, 60\} \text{ min} \quad (17)$$

L'horizon optimal $k_i^* = \arg \max_k \text{AUROC}(f_i, k)$ est appris indépendamment pour chaque type. La grille couvre les cascades rapides ($k = 2$ min : spikes de retry, queue overflow) comme les pannes

à développement lent ($k = 60$ min : fuites mémoire, remplissage de disque). Le choix définitif de la grille sera confirmé par l’analyse de la distribution des temps entre premiers symptômes et incident sur les données Chaos Mesh.

4.6 Sortie : Alerte EWAT

$$\text{Alert}(t) = (C_i, \hat{p}_i(t), k_i^*, \text{fiche}_{C_i}) \quad (18)$$

où fiche_{C_i} est la description interprétable produite à l’Étape 2 par SHAP.

4.7 Budget de latence

Pour un système d’early warning, la latence d’inférence est critique : si le pipeline met τ_{pipe} à produire une alerte pour un horizon k_i^* , le temps d’action réel pour le SRE est $k_i^* - \tau_{\text{pipe}}$.

Étape	Complexité	Latence cible
Étape 0 (MMD-RFF)	$O(nD)$	< 1 s
Étape 1 (STGCN forward)	$O(N^2 d_e)$	< 2 s
Étape 3 (précurseur f_i)	$O(k \cdot d)$	< 1 s
Total inférence		< 5 s

Les étapes 2 et 2b (typage, ontologie) s’exécutent offline et sont hors du chemin critique. Avec $k_{\min}^* = 2$ min et $\tau_{\text{pipe}} < 5$ s, le ratio est inférieur à 4 %, ce qui est acceptable. Cette contrainte sera validée par un benchmark de latence sur le cluster réel.

5. Protocole d’Évaluation

5.1 Falsification des hypothèses

Hyp.	Expérience	Métrique	Falsification
H1	Clustering, 5 graines, 5 splits	Silhouette held-out	< 0.3
H2	Drifts bénins intercalés, Chaos Mesh	ΔFPR à rappel constant	$p > 0.05$ (Student)
H3	Par type, $k \in \{2, 5, 10, 20, 30, 60\}$ min	AUROC vs. baseline	$< \text{baseline } \forall k$

Table 1: Protocole de falsification des trois sous-hypothèses.

5.2 Justification du seuil de silhouette

Le seuil de 0.3 est fondé sur la classification de Kaufman et Rousseeuw (1990) : en dessous de 0.25, la structure de clusters est considérée comme artificielle ; entre 0.25 et 0.50, la structure est faible mais présente ; au-dessus de 0.50, elle est raisonnable. Un seuil de 0.3 est donc conservateur pour un critère de falsification.

Le silhouette est mesuré sur des épisodes non vus pendant l’entraînement du réseau siamois, afin d’éviter le biais circulaire : le siamois étant entraîné pour séparer les types, le silhouette dans l’espace appris serait mécaniquement élevé sans cette précaution. Les 5 graines utilisent 5 splits train / test distincts (stratified k-fold sur les types d’injection), testant la robustesse de la structure et non seulement la stabilité de l’optimisation.

En complément du silhouette, la stabilité du nombre de clusters K est évaluée par le gap statistic (Tibshirani et al., 2001) et par le BIC d'un GMM dans l'espace des embeddings. La convergence des trois méthodes sur un même $K > 1$ constitue un support solide pour H1.

5.3 Ablation des features

L'ablation par modalité consiste à entraîner et évaluer avec M seul, T seul, L seul, puis toutes les paires, puis l'ensemble $M + T + L$, en mesurant l'impact sur le silhouette (H1) et l'AUROC (H3).

L'ablation par feature procède au retrait individuel de chacune des 17 dimensions (leave-one-out). Les features dont le retrait dégrade significativement les métriques sont identifiées par test de Wilcoxon signé sur les 5 splits.

L'analyse de redondance examine la matrice de corrélation inter-features en régime θ_{normal} . Les paires fortement corrélées ($|\rho| > 0.9$) sont candidates à l'élimination pour réduire d sans perte de performance.

6. Synthèse des Contributions

Contribution	Nature	Nouveauté
Séparation drift / anomalie par MMD-RFF avec look-through	Méthodologique	Aucun système RCA / EWAT actuel ne l'implémente
Ontologie empirique des anomalies Kubernetes	Empirique	Première validation sur cluster réel avec chaos engineering contrôlé
Relations causales par Transfer Entropy (estimateur KSG)	Algorithmique	Remplace Granger, inadapté aux systèmes non-stationnaires
Prédiction typée à horizon variable k_i^* par type	Applicative	Horizon optimal appris par type, non fixé globalement
Agrégation différenciée par type de métrique	Méthodologique	Sensibilité aux gray failures, évite la dilution par moyennage

Table 2: Contributions du projet EWAT.

6.1 Positionnement dans la littérature

Système	Sépare drift	Ontologie	Prédicatif	Interprétable
GrayScope (FSE'24)	×	×	×	✓
DynaCausal (ICSE'25)	×	×	×	×
RCACopilot (EuroSys'24)	×	×	×	✓
Chain-of-Event (FSE'24)	×	×	×	partiel
EWAT (ce travail)	objectif	objectif	objectif	objectif

Table 3: Positionnement par rapport aux méthodes SOTA 2024–2025. Les capacités d'EWAT sont des objectifs de conception, en l'absence de résultats expérimentaux.

6.2 Références

- Chandola, Banerjee et Kumar (2009). *Anomaly Detection: A Survey*. ACM Computing Surveys.
- Fu et al. (2025). *Intelligent RCA in Microservice Systems: A Survey*. ACM Computing Surveys.
- Gregg, B. (2013). *Systems Performance: Enterprise and the Cloud*. Prentice Hall.
- Hinder et al. (2024). *Concept drift in unsupervised data streams*. Frontiers in AI.
- Kaufman et Rousseeuw (1990). *Finding Groups in Data*. Wiley.
- Kraskov, Stögbauer et Grassberger (2004). *Estimating mutual information*. Physical Review E.
- Myrtollari et al. (2025). *Concept drift-aware anomaly detection for microservices on Kubernetes*. SSRN.
- Pham et al. (2024). *RCA for Microservices based on Causal Inference: How Far Are We?* ASE'24.
- GrayScope (2024). *Non-intrusive gray failure localization*. ACM FSE'24.
- Tibshirani, Walther et Hastie (2001). *Estimating the number of clusters via the gap statistic*. JRSS-B.
- Zamanzadeh Darban et al. (2024). *Deep Learning for Time Series AD: A Survey*. ACM Computing Surveys.